A Very Brief Introduction to Machine Learning With Applications to Communication Systems

Osvaldo Simeone, *Fellow, IEEE*

arXiv:1808.02342v4 [cs.IT] 5 Nov 2018

***Abstract*—Given the unprecedented availability of data and computing resources, there is widespread renewed interest in applying data-driven machine learning methods to problems for which the development of conventional engineering solutions is challenged by modelling or algorithmic deficiencies. This tutorial-style paper starts by addressing the questions of why and when such techniques can be useful. It then provides a high-level introduction to the basics of supervised and unsupervised learning. For both supervised and unsupervised learning, exemplifying applications to communication networks are discussed by distinguishing tasks carried out at the edge and at the cloud segments of the network at different layers of the protocol stack, with an emphasis on the physical layer.**

I. Introduction

After the "AI winter" of the 80s and the 90s, interest in the application of data-driven Artificial Intelligence (AI) techniques has been steadily increasing in a number of engineering fields, including speech and image analysis [1] and communications [2]. Unlike the logic-based expert systems that were dominant in the earlier work on AI (see, e.g., [3]), the renewed confidence in data-driven methods is motivated by the successes of pattern recognition tools based on machine learning. These tools rely on decades-old algorithms, such as backpropagation [4], the Expectation Maximization (EM) algorithm [5], and Q-learning [6], with a number of modern algorithmic advances, including novel regularization techniques and adaptive learning rate schedules (see review in [7]). Their success is built on the unprecedented availability of data and computing resources in many engineering domains.

While the new wave of promises and breakthroughs around machine learning arguably falls short, at least for now, of the requirements that drove early AI research [3], [8], learning algorithms have proven to be useful in a number of important applications – and more is certainly on the way.

King's College London, United Kingdom (email: osvaldo.simeone@kcl.ac.uk). This work has received funding from the European Research Council (ERC) under the European Union Horizon 2020 research and innovation program (grant agreement 725731).

This paper provides a very brief introduction to key concepts in machine learning and to the literature on machine learning for communication systems. Unlike other review papers such as [9]–[11], the presentation aims at highlighting conditions under which the use of machine learning is justified in engineering problems, as well as specific classes of learning algorithms that are suitable for their solution. The presentation is organized around the description of general technical concepts, for which an overview of applications to communication networks is subsequently provided. These applications are chosen to exemplify general design criteria and tools and not to offer a comprehensive review of the state of the art and of the historical progression of advances on the topic.

We proceed in this section by addressing the question "What is machine learning?", by providing a taxonomy of machine learning methods, and by finally considering the question "When to use machine learning?".

A. What is Machine Learning?

In order to fix the ideas, it is useful to introduce the machine learning methodology as an alternative to the conventional engineering approach for the design of an algorithmic solution. As illustrated in Fig. 1(a), the conventional engineering design flow starts with the *acquisition of domain knowledge*: The problem of interest is studied in detail, producing a *mathematical model* that capture the *physics* of the set-up under study. Based on the model, an *optimized algorithm* is produced that offers *performance guarantees* under the assumption that the given physics-based model is an accurate representation of reality.

As an example, designing a decoding algorithm for a wireless fading channel under the conventional engineering approach would require the development, or the selection, of a physical model for the channel connecting transmitter and receiver. The solution would be obtained by tackling an optimization problem, and it would yield optimality guarantees under the given channel model. Typical example of channel models include Gaussian and fading channels (see, e.g., [12]).

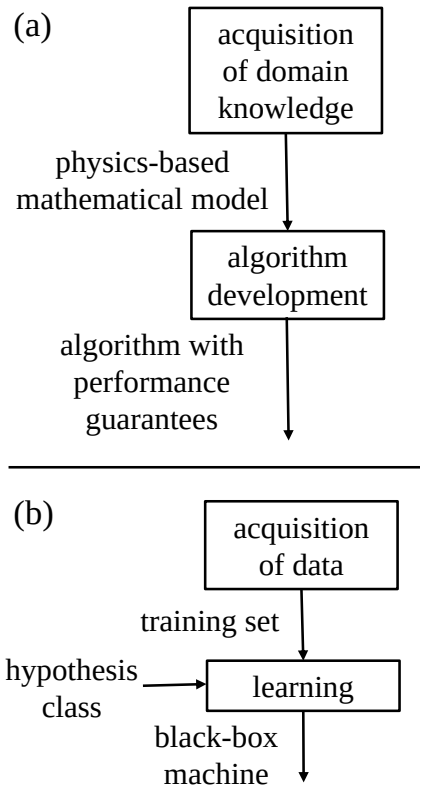

Fig. 1. (a) Conventional engineering design flow; and (b) baseline machine learning methodology.

In contrast, in its most basic form, the machine learning approach substitutes the step of acquiring domain knowledge with the potentially easier task of collecting a sufficiently large number of examples of desired behaviour for the algorithm of interest. These examples constitute the *training set*. As seen in Fig. 1(b), the examples in the training set are fed to a learning algorithm to produce a trained "machine" that carries out the desired task. Learning is made possible by the choice of a set of possible "machines", also known as the *hypothesis class*, from which the learning algorithm makes a selection during training. An example of an hypothesis class is given by a neural network architecture with learnable synaptic weights. Learning algorithms are generally based on the optimization of a performance criterion that measures how well the selected "machine" matches the available data.

For the problem of designing a channel decoder, a machine learning approach can hence operate even in the absence of a well-established channel model. It is in fact enough to have a sufficiently large number of examples of received signals – the inputs to the decoding machine – and transmitted messages – the desired outputs of the decoding machine – to be used for the training of a given class of decoding functions [13].

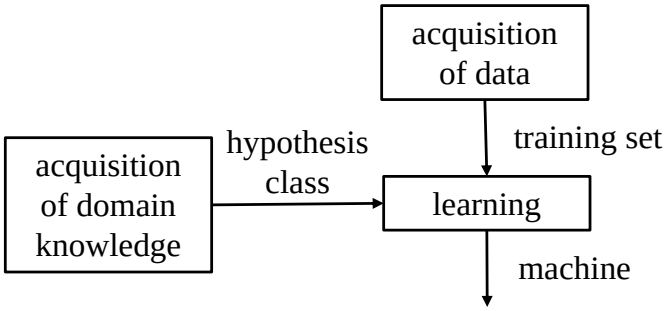

Fig. 2. Machine learning methodology that integrates domain knowledge during model selection.

Moving beyond the basic formulation described above, machine learning tools can *integrate available domain knowledge* in the learning process. This is indeed the key to the success of machine learning tools in a number of applications. A notable example is image processing, whereby knowledge of the translational invariance of visual features is reflected in the adoption of convolutional neural networks as the hypothesis class to be trained. More generally, as illustrated in Fig. 2, domain knowledge can dictate the choice of a specific hypothesis class for use in the training process. Examples of applications of this idea to communication systems, including to the problem of decoding, will be discussed later in the paper.

B. Taxonomy of Machine Learning Methods

There are three main classes of machine learning techniques, as discussed next.

- *Supervised learning*: In supervised learning, the training set consists of pairs of input and desired output, and the goal is that of learning a mapping between input and output spaces. As an illustration, in Fig. 3(a), the inputs are points in the two-dimensional plane, the outputs are the labels assigned to each input (circles or crosses), and the goal is to learn a binary classifier. Applications include the channel decoder discussed above, as well as email spam classification on the basis of examples of spam/ non-spam emails.
- *Unsupervised learning*: In unsupervised learning, the training set consists of unlabelled inputs, that is, of inputs without any assigned desired output. For instance, in Fig. 3(b), the inputs are again points in the two-dimensional plane, but no indication is provided by the data about the corresponding desired output. Unsupervised learning generally aims at discovering properties of the mechanism generating the data. In the example of Fig. 3(b), the goal of unsupervised learning is to cluster together

input points that are close to each other, hence assigning a label – the cluster index – to each input point (clusters are delimited by dashed lines). Applications include clustering of documents with similar topics. It is emphasized that clustering is only one of the learning tasks that fall under the category of unsupervised learning (see Sec. V).

- *Reinforcement learning*: Reinforcement learning lies, in a sense, between supervised and unsupervised learning. Unlike unsupervised learning, some form of supervision exists, but this does not come in the form of the specification of a desired output for every input in the data. Instead, a reinforcement learning algorithm receives feedback from the environment only after selecting an output for a given input or observation. The feedback indicates the degree to which the output, known as action in reinforcement learning, fulfils the goals of the learner. Reinforcement learning applies to sequential decision making problems in which the learner interacts with an environment by sequentially taking actions – the outputs – on the basis of its observations – its inputs – while receiving feedback regarding each selected action.

Most current machine learning applications fall in the supervised learning category, and hence aim at learning an existing pattern between inputs and outputs. Supervised learning is relatively well-understood at a theoretical level [14], [15], and it benefits from well-established algorithmic tools. Unsupervised learning has so far defied a unified theoretical treatment [16]. Nevertheless, it arguably poses a more fundamental practical problem in that it directly tackles the challenge of learning by direct observation without any form of explicit feedback. Reinforcement learning has found extensive applications in problems that are characterized by clear feedback signals, such as win/lose outcomes in games, and that entail searches over large trees of possible action-observation histories [17], [18].

This paper only covers supervised and unsupervised learning. Reinforcement learning requires a different analytical framework grounded in Markov Decision Processes and will not be discussed here (see [17]). For a broader discussion on the technical aspects of supervised and unsupervised learning, we point to [19] and references therein.

C. When to Use Machine Learning?

Based on the discussion in Sec. I-A, the use of a machine learning approach in lieu of a more conventional engineering design should be justified on a case-by-case basis on the basis of its suitability and potential advantages. The following criteria, inspired by [20], offer useful guidelines on the type of engineering tasks that can benefit from the use of machine learning tools.

1. *The traditional engineering flow is not applicable or is undesirable due to a model deficit or to an algorithm deficit* [21].

- With a *model deficit*, no physics-based mathematical models exist for the problem due to insufficient domain knowledge. As a result, a conventional model-based design is inapplicable.
- With an *algorithm deficit*, a well-established mathematical model is available, but existing algorithms optimized on the basis of such model are too complex to be implemented for the given application. In this case, the use of hypothesis classes including efficient "machines", such as neural network of limited size or with tailored hardware implementations (see, e.g., [22], [23] and references therein), can yield lower-complexity solutions.

2. *A sufficiently large training data sets exist or can be created.*

3. *The task does not require the application of logic, common sense, or explicit reasoning based on background knowledge.*

4. *The task does not require detailed explanations for how the decision was made.* The trained machine is by and large a black box that maps inputs to outputs. As such, it does not provide direct means to ascertain why a given output has been produced in response to an input, although recent research has made some progress on this front [24]. This contrasts with engineered optimal solutions, which can be typically interpreted on the basis of physical performance criteria. For instance, a maximum likelihood decoder chooses a given output because it minimizes the probability of error under the assumed model.

5. *The phenomenon or function being learned is stationary for a sufficiently long period of time.* This is in order to enable data collection and learning.

6. *The task has either loose requirement constraints, or, in the case of an algorithm deficit, the required performance guarantees can be provided via numerical simulations.* With the conventional engineering approach, theoretical performance guarantees can be obtained that are backed by a physics-based mathematical model. These guarantees can be relied upon insofar as the model is trusted to be an accurate representation of reality. If a machine learning approach is used to address an algorithm deficit and a physics-based model is available, then numerical results may be sufficient in order to compute satisfactory performance measures. In

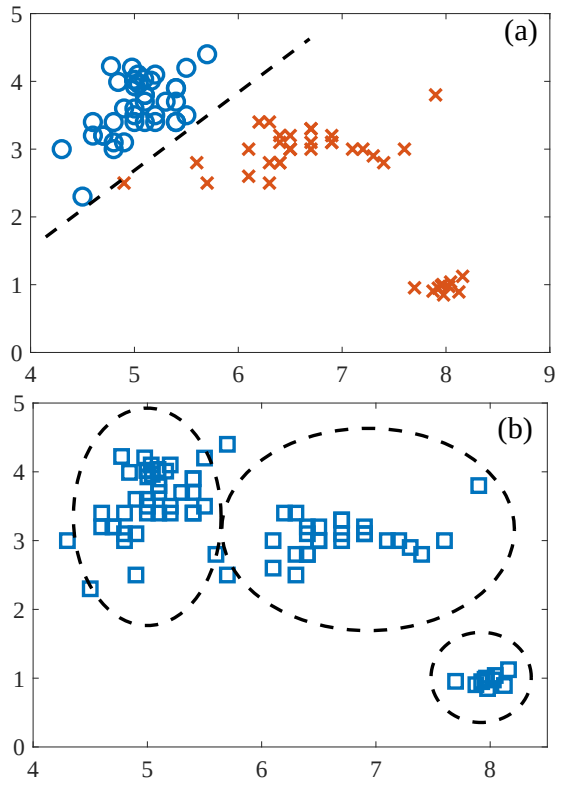

Fig. 3. Illustration of (a) supervised learning and (b) unsupervised learning.

contrast, weaker guarantees can be offered by machine learning in the absence of a physics-based model. In this case, one can provide performance bounds only under the assumptions that the hypothesis class is sufficiently general to include "machines" that can perform well on the problem and that the data is representative of the actual data distribution to be encountered at runtime (see, e.g., [19][Ch. 5]). The selection of a biased hypothesis class or the use of an unrepresentative data set may hence yield strongly suboptimal performance.

We will return to these criteria when discussing applications to communication systems.

II. Machine Learning for Communication Networks

In order to exemplify applications of supervised and unsupervised learning, we will offer annotated pointers to the literature on machine learning for communication systems. Rather than striving for a comprehensive, and historically minded, review, the applications and references have been selected with the goal of illustrating key aspects regarding the use of machine learning in engineering problems.

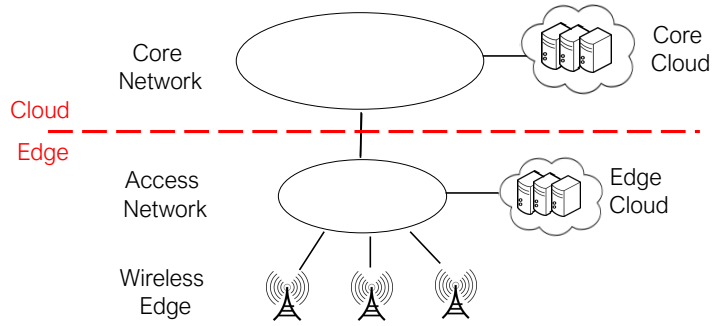

Fig. 4. A generic cellular wireless network architecture that distinguishes between edge segment, with base stations, access points, and associated computing resources, and cloud segment, consisting of core network and associated cloud computing platforms.

Throughout, we focus on tasks carried out at the network side, rather than at the users, and organize the applications along two axes. On one, with reference to Fig. 4, we distinguish tasks that are carried out at the *edge* of the network, that is, at the base stations or access points and at the associated computing platforms, from tasks that are instead responsibility of a centralized *cloud* processor connected to the core network (see, e.g., [25]). The edge operates on the basis of timely local information collected at different layers of the protocol stack, which may include all layers from the physical up to the application layer. In contrast, the centralized cloud processes longer-term and global information collected from multiple nodes in the edge network, which typically encompasses only the higher layers of the protocol stack, namely networking and application layers. Examples of data that may be available at the cloud and at the edge can be found in Table I and Table II, respectively.

As a preliminary discussion, it is useful to ask which tasks of a communication network, if any, may benefit from machine learning through the lens of the criteria reviewed in Sec. I-C. First, as seen, there should be either a model deficit or an algorithm deficit that prevents the use of a conventional model-based engineering design. As an example of model deficit, proactive resource allocation that is based on predictions of human behaviour, e.g., for caching popular contents, may not benefit from well-established and reliable models, making a data-driven approach desirable (see, e.g., [26], [27]). For an instance of algorithm deficit, consider the problem of channel decoding for channels with known and accurate models based on which the maximum likelihood decoder entails an excessive complexity.

Assuming that the problem at hand is characterized by model or algorithm deficits, one should then consider the rest of the criteria discussed in Sec. I-C. Most are

TABLE I
EXAMPLES OF DATA AVAILABLE AT THE EDGE SEGMENT OF A COMMUNICATION NETWORK

Layer	Data
Physical	Baseband signals, channel state information
Medium Access Control/ Link	Throughput, FER, random access load and latency
Network	Location, traffic loads across services, users' device types, battery levels
Application	Users' preferences, content demands, computing loads, QoS metrics

TABLE II
EXAMPLES OF DATA AVAILABLE AT THE CLOUD SEGMENT OF A COMMUNICATION NETWORK

Layer	Data
Network	Mobility patterns, network-wide traffic statistics, outage rates
Application	User's behaviour patterns, subscription information, service usage statistics, TCP/IP traffic statistics

typically satisfied by communication problems. Indeed, for most tasks in communication networks, it is possible to collect or generate training data sets and there is no need to apply common sense or to provide detailed explanations for how a decision was made.

The remaining two criteria need to be checked on a case-by-case basis. First, the phenomenon or function being learned should not change too rapidly over time. For example, designing a channel decoder based on samples obtained from a limited number of realizations of a given propagation channel requires the channel is stationary over a sufficiently long period of time (see [28]).

Second, in the case of a model deficit, the task should have some tolerance for error in the sense of not requiring provable performance guarantees. For instance, the performance of a decoder trained on a channel lacking a well-established channel model, such as a biological communication link, can only be relied upon insofar as one trusts the available data to be representative of the complete set of possible realizations of the problem under study. Alternatively, under an algorithm deficit, a physics-based model, if available, can be possibly used to carry out computer simulations and obtain numerical performance guarantees.

In Sec. IV and Sec. VI, we will provide some pointers to specific applications to supervised and unsupervised learning, respectively.

III. SUPERVISED LEARNING

As introduced in Sec. I, supervised learning aims at discovering patterns that relate inputs to outputs on the basis of a training set of input-output examples. We can distinguish two classes of supervised learning problems depending on whether the outputs are continuous or discrete variables. In the former case, we have a *regression* problem, while in the latter we have a *classification* problem. We discuss the respective goals of the two problems next. This is followed by a formal definition of classification and regression, and by a discussion of the methodology and of the main steps involved in tackling the two classes of problems.

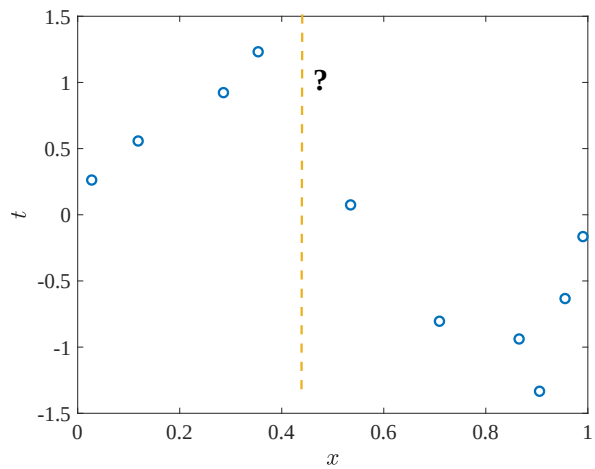

Fig. 5. Illustration of the supervised learning problem of regression: Given input-output training examples (x_n, t_n), with $n = 1, ..., N$, how should we predict the output t for an unobserved value of the input x?

A. Goals

As illustrated in Fig. 5, in a regression problem, we are given a training set $\mathcal{D}$ of N training points (x_n, t_n), with $n = 1, ..., N$, where the variables x_n are the inputs, also known as covariates, domain points, or explanatory variables; while the variables t_n are the outputs, also known as dependent variables, labels, or responses. In regression, the outputs are continuous variables. The problem is to predict the output t for a new, that is, as of yet unobserved, input x.

As illustrated in Fig. 6, classification is similarly defined with the only caveat that the outputs t are discrete